SIMATS ENGINEERING

ASSESSMENT TOOL III — TIME-SERIES & GEOSPATIAL ANALYSIS REPORT

Course: Data Handling and Visualization	**Code:** DSA0603	**CO:** CO4
Duration: 60 min	**Total Marks:** 50	Reg No: 192324285 Name: Surya JK

COURSE OUTCOME COVERED

Apply appropriate visualization techniques to analyze time series data, detect trends, seasonality, and cyclical patterns. (BL3)
CO3 Construct and interpret geospatial visualizations, including static, cartogram-based, and interactive representations. (BL3)
Activity Tool : Time-Series & Geospatial Analysis Report
Weightage: 20%

Code of Conduct:

I Surya JK / 192324285 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Data Understanding & Preparation 25%	Time-Series/Geospatial Analysis 25%	Application of Visualization Techniques 20%	Report Organization 15%	Accuracy 10%	Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

TIME-SERIES & GEOSPATIAL ANALYSIS REPORT
QUESTIONS WITH ANSWERS

Note: Questions 1–4 are in R and Question 5 is in Python, as specified in the source document. The supplied PDF identifies the datasets and tasks; the answers below follow those requirements. □filecite□turn2file0□L47-L65□

Question 1: Individual Time Series — Smoothing & Detrending

Question: Using Nile.csv, convert value into an R ts object (start = 1871, frequency = 1), plot the raw annual flow, overlay a 5-year moving-average smoother, fit a linear trend (value ~ time), plot detrended residuals in a second panel, and discuss what the smoothed series and residual plot reveal about a level shift/change point.

Answer: The smoothed Nile series makes the major level change around 1898 visible: observations are generally at a higher level before the change and a lower level afterward. The residual plot removes the broad linear trend but still shows systematic structure around the level shift, indicating that a single straight-line trend does not fully describe the series. This is consistent with the well-known level shift in the Nile data.

Code:

```
nile <- read.csv("Nile.csv")
nile_ts <- ts(nile$value, start = 1871, frequency = 1)

ma5 <- stats::filter(nile_ts, rep(1/5, 5), sides = 2)

fit <- lm(as.numeric(nile_ts) ~ time(nile_ts))
resid_ts <- ts(residuals(fit), start = start(nile_ts),
               frequency = frequency(nile_ts))

par(mfrow = c(2, 1))
plot(nile_ts, type="l", main="Nile Flow with 5-Year Moving Average",
     xlab="Year", ylab="Flow")
lines(ma5, lwd=2)

plot(resid_ts, type="l", main="Detrended Nile Residuals",
     xlab="Year", ylab="Residual")
```

Question 2: Seasonal Decomposition (STL) & Forecasting

Question: Using AirPassengers.csv, create a monthly ts object (start = c(1949,1), frequency = 12), apply STL with s.window = 'periodic', plot the components, decide whether additive or multiplicative decomposition is better, then fit ETS or auto.arima() and produce a 24-month forecast with prediction intervals.

Answer: Multiplicative decomposition is more appropriate for AirPassengers because the seasonal amplitude grows as the overall passenger level increases. The forecast should preserve the historical annual seasonal pattern, with recurring peaks and troughs, while the long-term level continues upward. Prediction intervals widen with the forecast horizon because uncertainty increases further into the future.

Code:

```
library(forecast)

ap <- read.csv("AirPassengers.csv")
ap_ts <- ts(ap$passengers, start=c(1949,1), frequency=12)

# STL on log scale represents multiplicative seasonality
stl_fit <- stl(log(ap_ts), s.window="periodic")
plot(stl_fit)

fit <- auto.arima(ap_ts)
fc <- forecast(fit, h=24)
autoplot(fc) +
  labs(title="24-Month AirPassengers Forecast",
       x="Year", y="Passengers")
```

Question 3: Choropleth Map & Interactive Geospatial Plot

Question: Part (a): Join USArrests.csv to US state boundaries and create a choropleth of Murder rate by state using a sequential color scale. Identify the state(s) with the highest murder rate and describe regional clustering. Part (b): Use leaflet with quakes.csv to plot earthquake locations as circle markers, size by magnitude and color by depth, add popups, and describe clustering of high-magnitude events.

Answer: Part (a): Georgia has the highest Murder rate in the classic USArrests data (17.4 per 100,000). The map shows higher murder rates concentrated more strongly in parts of the South, while many northern and western states have lower values. Part (b): The earthquake points are spatially clustered in the Fiji-region study area rather than being uniformly distributed. High-magnitude events can be identified by their larger markers; the popup should report magnitude and depth for each event.

Code:

```
library(ggplot2)
library(maps)
library(leaflet)

ar <- read.csv("USArrests.csv")
ar$state <- tolower(rownames(ar))

states <- map_data("state")
map_df <- merge(states, ar, by.x="region", by.y="state")

ggplot(map_df, aes(long, lat, group=group, fill=Murder)) +
  geom_polygon(color="white") +
  coord_fixed(1.3) +
  scale_fill_gradient(low="lightyellow", high="red") +
  labs(title="US Murder Rate by State",
       fill="Murder Rate",
       caption="Source: USArrests.csv")

# Interactive earthquakes
q <- read.csv("quakes.csv")
pal <- colorNumeric("viridis", domain=q$depth)

leaflet(q) |>
  addTiles() |>
  addCircleMarkers(~long, ~lat,
                   radius=~mag,
                   color=~pal(depth),
                   popup=~paste("Magnitude:", mag,
                                "<br>Depth:", depth))
```

Question 4: Cartogram with Map Projections

Question: Using state_x77.csv, join the data to US state boundaries and construct a population-weighted cartogram using cartogram_cont() or cartogram_ncont(). Render it beside a standard population choropleth using two projections, such as an Albers equal-area projection and longitude/latitude, and discuss the visual differences.

Answer: The population-weighted cartogram enlarges states with large populations and shrinks states with small populations, so the visual impression of geographic size changes substantially compared with a true-area map. California, Texas, New York and other highly populated states become visually more prominent, while sparsely populated states occupy less cartogram area. In the standard map, the Albers equal-area projection gives a more suitable shape/area representation for comparing US states, whereas a simple longitude/latitude view introduces stronger distortion toward the north and south. Thus, projection changes the apparent shape, while the cartogram intentionally changes area according to population.

Code:

```
library(sf)
library(ggplot2)
library(cartogram)

st <- read.csv("state_x77.csv")
# Obtain state polygons and convert to sf
states <- st_as_sf(map("state", plot=FALSE, fill=TRUE))
states$state <- tolower(states$ID)

st$state <- tolower(st$state)
joined <- merge(states, st, by="state")

# Population-weighted cartogram
cart <- cartogram_cont(joined, "Population", itermax=5)

p1 <- ggplot(joined) +
  geom_sf(aes(fill=Population)) +
  coord_sf(crs=5070) +
  labs(title="Standard Population Choropleth")
```

```
p2 <- ggplot(cart) +
  geom_sf(aes(fill=Population)) +
  labs(title="Population-Weighted Cartogram")

p1
p2
```

Question 5: 3D Geospatial Heatmap

Question: Using Python and volcano_elevation.csv, pivot x, y and elevation into a 2D elevation grid, create an interactive Plotly 3D surface heatmap with hover elevation values, also create a 2D top-down heatmap, and compare what each view reveals.

Answer: The 3D surface view reveals the volcano's terrain as an actual surface: peaks, slopes and valleys can be distinguished through height as well as color. The 2D heatmap is easier for comparing elevation values across the whole grid at a glance, but it does not show the physical relief and slope relationships as directly as the 3D surface. Together, the two views provide complementary information about Maunga Whau's elevation structure.

Code:

```
import pandas as pd
import plotly.graph_objects as go
import matplotlib.pyplot as plt

df = pd.read_csv("volcano_elevation.csv")

grid = df.pivot(index="y", columns="x", values="elevation")
x = grid.columns.values
y = grid.index.values
z = grid.values

fig3d = go.Figure(
    data=[go.Surface(x=x, y=y, z=z, colorscale="Earth",
                     hovertemplate="x=%{x}<br>y=%{y}<br>"
                                   "Elevation=%{z}<extra></extra>")]
)
fig3d.update_layout(
    title="Maunga Whau 3D Elevation Surface",
    scene=dict(xaxis_title="X", yaxis_title="Y",
               zaxis_title="Elevation")
)
fig3d.show()

fig2d = go.Figure(
    data=[go.Heatmap(x=x, y=y, z=z, colorscale="Earth")]
)
fig2d.update_layout(
    title="Maunga Whau 2D Elevation Heatmap",
    xaxis_title="X", yaxis_title="Y"
)
fig2d.show()
```